HTML

Structure and Content of a Document

tag, element, attribute, block/inline, DOM

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

doc. Ing. Radek Burget, Ph.D.

burgetr@vut.cz

Ing. Jiří Hynek, Ph.D.

hynek@vut.cz

Motivation

*To present the **content** of a document,*
*describe its **structure**,*
*and the **meaning** of its individual parts.*

Motivation – Example

Markup Languages

- A tool for creating documents, defining structure and meaning

- **Traditional programming languages** – dominated by structured text (source code), general text appears as ***strings***

```
if(strcmp("I am not in danger, Skyler.", msg) == 0)
  printf("I am the danger.");
```

- **Markup languages** – dominated by general text, with structured sections inserted using ***tags***.

```
You got one part of that wrong,
  <a href="mercury.html">this</a>
  is not <a href="crystal.html">meth</a>
```

Types of Markup Languages

- different in syntax and usage

- **Data serialization, configuration** – **XML**, **JSON**, YAML, TOML, INI, …
- **Printed documents**
  - TeX (LaTeX) – publishing
  - Rich Text Format (RTF) – office documents
- **Electronic manuals** – nroff, troff
- **Electronic documents**
  - **Markdown**, Wiki (DokuWiki, MediaWiki), … – using intuitive symbols
- **Web pages**
  - XML family (**XHTML**, **SVG**, MathML, …)
  - **HTML**

Markdown

```markdown
# 🧙 Severus Snape
> *“Always.”*

## Profile
- **Full Name:** Severus Tobias Snape
- **House:** Slytherin
- **Blood Status:** Half-blood

## Roles & Affiliations
- Potions Master, Headmaster of Hogwarts
- Member of the Order of the Phoenix
- Former Death Eater

## Skills
- Expert in Potions and Dark Arts
- Skilled in Occlumency & Legilimency
- Inventor of spells (*e.g. Sectumsempra*)

## Portrayed By
Alan Rickman (2001–2011)
```

```
====== 🧙 Severus Snape ======
> '''"Always."'''

===== Profile =====
  * **Full Name:** Severus Tobias Snape
  * **House:** Slytherin
  * **Blood Status:** Half-blood

===== Roles & Affiliations =====
  * Potions Master, Headmaster of Hogwarts
  * Member of the Order of the Phoenix
  * Former Death Eater

===== Skills =====
  * Expert in Potions and Dark Arts
  * Skilled in Occlumency & Legilimency
  * Inventor of spells (e.g. //Sectumsempra//)

===== Portrayed By =====
Alan Rickman (2001–2011)
```

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Severus Snape Profile</title>
</head>
<body>
  <h1>🧙 Severus Snape</h1>
  <blockquote><em>"Always."</em></blockquote>

  <h2>Profile</h2>
  <ul>
    <li><strong>Full Name:</strong> Severus Tobias Snape</li>
    <li><strong>House:</strong> Slytherin</li>
    <li><strong>Blood Status:</strong> Half-blood</li>
  </ul>

  <h2>Roles & Affiliations</h2>
  <ul>
    <li>Potions Master, Headmaster of Hogwarts</li>
    <li>Member of the Order of the Phoenix</li>
    <li>Former Death Eater</li>
```

HTML Language

Introduction, Principles

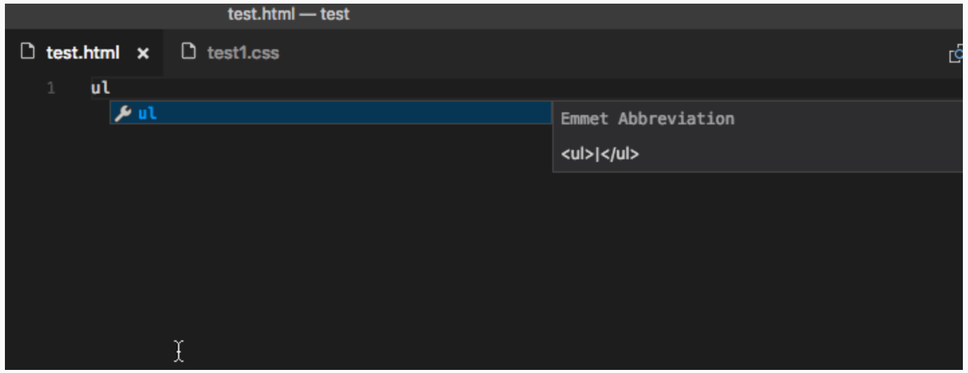
HTML

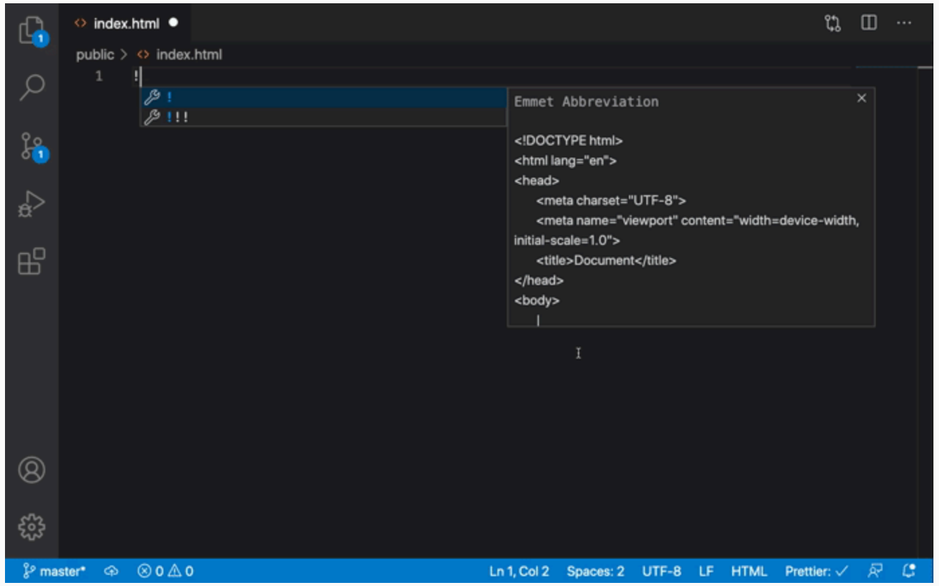
- HTML = *Hypertext Markup Language*
- standard language for creating hypertext documents

- **1994**: **W3C** is founded (*World Wide Web Consortium*) – main organization for web standards
  - **HTML 4.01** – insufficient (multimedia, semantics, inconsistent implementation, …)
- **2000–2004**: W3C promotes XHTML – impractical for the common web (strict syntax, complexity, …)
- **2004**: **WHATWG** is founded (*Web Hypertext Application Technology Working Group*)
  - reaction to stagnation in HTML development
  - develops **HTML5** in parallel --> confusion
- **2014**: WHATWG introduces **HTML Living Standard**
  - unnumbered, continuously updated HTML
- **2019** 🥳 : Agreement between W3C and WHATWG
  - WHATWG becomes the official maintainer of HTML and DOM
  - W3C focuses on CSS, accessibility, and other technologies

HTML Document

- text document
  - MIME type text/html
  - usually with extension .html or .htm
- character encoding: **UTF-8**
  - old: ISO-8859-2, windows-1250, …
- popular editors, IDEs: **VSCode**, **WebStorm**, …





HTML Document: Basic Structure

- contains text, embedded tags, and character entities

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Document</title>
  </head>
  <body>
    <!-- Let's greet the new students -->
    <p>Hello! You shall not pass! &#x1F9D9;</p>
    <a href="https://www.fit.vut.cz/study/course/ITW/">ITW</a>
  </body>
</html>
```

- result:

Hello! You shall not pass! 🧙 ITW

HTML Document: Basic Structure

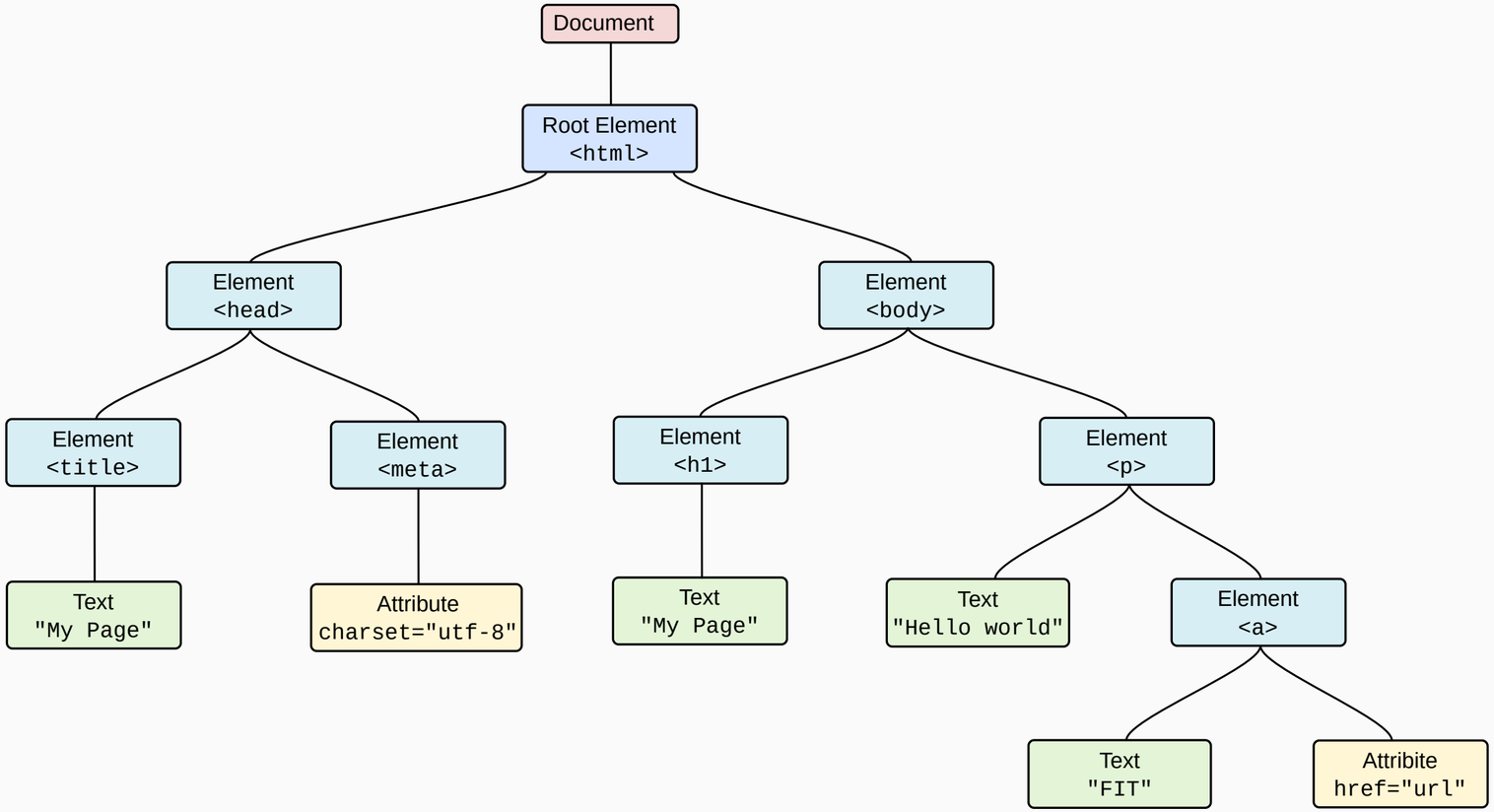
- **DOCTYPE** – instruction for the browser, document type declaration

```
<!DOCTYPE html>

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
  "http://www.w3.org/TR/html4/strict.dtd">

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

- **`<html lang="en">`** – root element and document language
- **`<head>`** – document header, metadata for the browser
  - `<meta charset="UTF-8">` – character encoding
  - `<title>Document</title>` – page title shown in browser tab
  - … (see further)
- **`<body>`** – document body, content shown to the user

HTML Document: DOM

- document forms an **tree** consisting of **nodes** (element, attribute, comment, text, …)



- CTRL+SHIFT+I / Inspector/Elements tab
- right-click on element / Inspect element

HTML Element

- part of a document delimited by **tags** (opening and closing)

```
<p>Element content</p>

<div class="menu" id="mainmenu">
  Element content<br> More element content.
</div>

<div>Some <em>emphasized</em> text.</div>
```

- consists of name, attributes, and content

- some elements may have only a start tag `<li>`, `<p>` (× XHTML)
- some elements may also lack content `<br>`, `<img>` (so-called void element)
- **case insensitive** – letter case does not matter (× XHTML)

HTML Element: Definition

- HTML specification defines existing elements and their properties:
  - semantics (meaning, what it represents)
  - context where it can be used
  - allowed content (other elements, text)
  - allowed attributes
  - possibility to omit closing tag

- example definition of the `<li>` element: **WHATWG**

- document validity: **online W3C validator**

HTML Attribute

- so-called ***content attributes*** – written in HTML as `name="value"`
  - express the initial state of elements
  - defined in element specifications
- **global attributes** – usable in every element
  - **`id`** – unique identifier (unique in document)
  - **`class`** – class
  - **`style`** – CSS inline styles
  - **`title`** – tooltip
  - **`lang`** – content language
  - **`hidden`** – hidden element
  - **`tabindex`** – tab activation order
  - more…
- custom attributes **`data-*`** (extensibility)

HTML Document: Comments

- **notes in the HTML document** not meant to be displayed by the browser
  - author comments
  - **remain in the source code!**

```
<!-- this is my plagiarized project -->
<html>
```

- **(server side includes)**
  - simple server-side scripting mechanism
  - inserting content from other files, running commands when generating the page on the server
  - must be supported on the server (e.g. Apache, nginx)

```
<!--#include file="header.html" -->
```

HTML Document: Text, Whitespace

- merged into one, lines are wrapped dynamically when rendering

```
<body>
  Hello!
  You shall not pass!

  &#x1F9D9;

  One Ring to rule them all, One Ring to find them, One Ring to bring them all and in the
</body>
```

```
Hello!
   You shall not pass!

   🧙

   One Ring to rule them all, One Ring to find them, One Ring to bring the
```

HTML Document: Character Entities

- for inserting special characters: **&code**

Entity	Character
`<`	<
`>`	>
`&`	&
`®`	®
`©`	©
`€`	€
` `	" " (non-breaking space)
`🧙`	🧙

HTML Document: Appearance

- **HTML language is not intended for defining appearance!** (× older HTML versions)

- **CSS language** (*Cascading Style Sheets*)
  - visual appearance of elements using **CSS properties** (`color`, `font-weight`, `margin`, …)

```
<h2>ITW</h2>
<p>Hello! <b>You</b> shall <span style="color: red">not</span> pass! &#x1F9D9;</p>
```

ITW
Hello! **You** shall not pass! 🧙

- browser provides default appearance for known HTML elements (<h2>, <b>, <p>, …)
- can be overridden with custom style rules (e.g. `<span style="color: red">`, **…**)

HTML Document: Purpose

- present **content** along with **structure** and **meaning**
- it is linked with other HTML documents via hypertext links – forms the web
- defines relationships with other document types
  - images, multimedia, style sheets, fonts, JavaScript

Document Content

block, inline, and inline-block elements

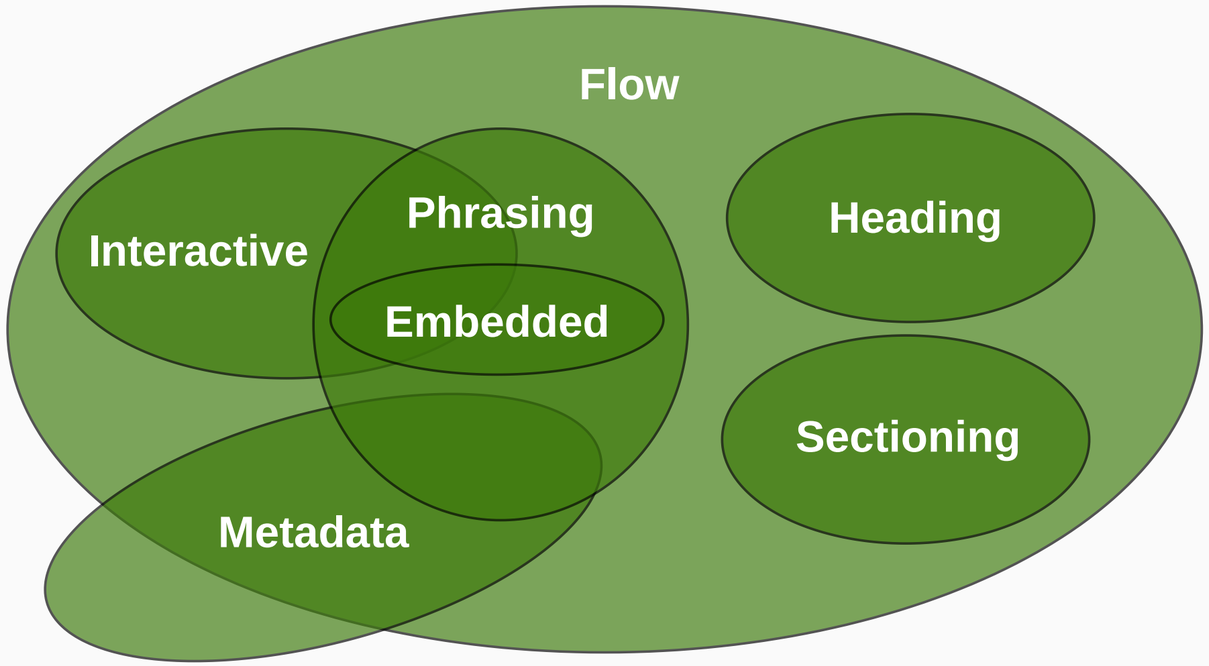
Document Content

- consists of the <body> element and its nested elements
- we create content and describe the structure and meaning of document parts

Type	Selected Example Elements
Sections & Containers	`div`, `section`, `article`, `nav`, `aside`
Headings	`h1` – `h6`
Lists	`ul`, `ol`, `li`, `dl`, `dt`, `dd`
Tables	`table`, `tr`, `td`, `th`, `thead`, `tbody`, `tfoot`
Text Elements	`p`, `span`, `strong`, `em`, `br`
Multimedia	`img`, `audio`, `video`, `svg`, `canvas`
Forms	`form`, `input`, `label`, `textarea`, `button`, `select`
Interactive	`a`, `button`, `details`, `summary`
Scripting	`script`, `template`

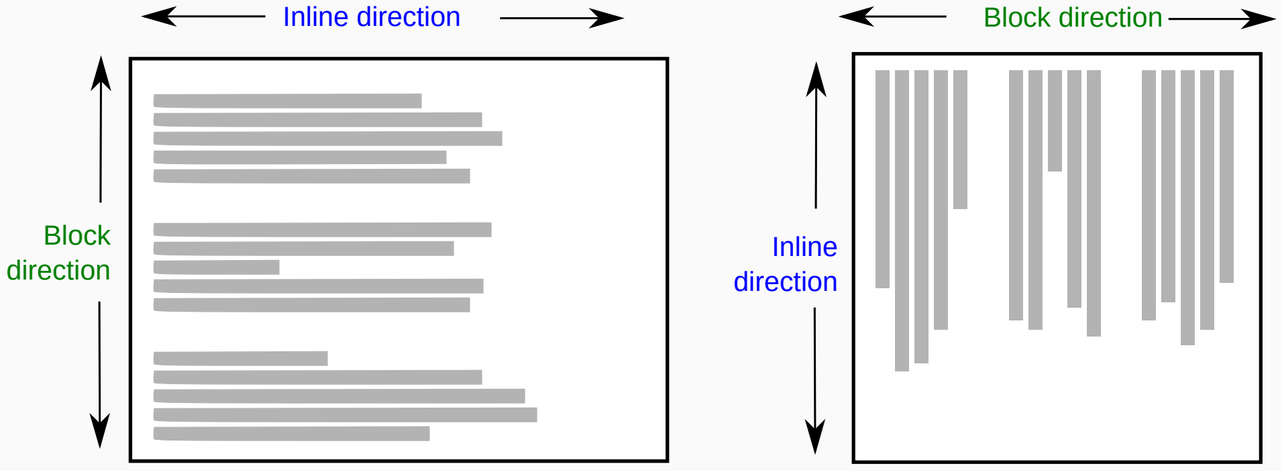
Flow Content

- various types of elements – according to specification, so-called ***flow content***
  - content elements, *“most elements used in the body of documents and applications”*



Normal Flow (CSS)

- normal flow in the document
- default way of laying out elements on the page, unless specified otherwise by styles
- formatting contexts:
  - **block** (*block formatting context*) – stacked vertically
  - **inline** (*inline formatting context*) – placed side by side
- for some Asian languages, the direction is reversed (so-called vertical writing mode)



Horizonal writing mode (Vertical writing mode)

Block Elements

- so-called ***block-level content*** (e.g. h2 or p)
- elements are stacked vertically, affect text flow and cause line breaks
- take up the full width of the parent (unless styled otherwise – see CSS lecture)
- width, height, margins (`margin`, `padding`) can be changed

```
Hello! <h2>You</h2> shall <p style="width: 200px;">not</p> pass! &#x1F9D9;
```

Hello! YOU shall not pass! 🧙

Inline Elements

- so-called ***inline-level content*** (e.g. `b` or `i`)
- elements are placed side by side, do not affect text flow, do not cause line breaks
- width is usually determined by content
- width and height cannot be changed

```
Hello! <b>You</b> shall <i style="width: 200px;">not</i> pass! &#x1F9D9;
```

Hello! **You** shall *not* pass! 🧙

- margins (`margin`, `padding`) can be changed, but vertical margins do not affect surrounding lines

Block vs. Inline Elements

Inline-block Elements

- a special type of inline element, called ***inline-block***
- behave like inline (can be in a line with text)
- but respect dimensions (width, height) and have block box model (`margin`, `padding`)
- mainly used for elements that should be in a line but also have fixed size
  - images, multimedia, forms
  - `img`, `svg`, `canvas`, `video`, `iframe`, `input`, `button`, ...

```html
<p>
  Hello! <img width="50" src="gandalf.png" alt="Gandalf">
  You shall not pass!. <img width="100" src="gandalf.png" alt="Gandalf">
</p>
```

Inline-block Elements

Comparison

	block	inline	inline-block
element arrangement	stacked vertically	side by side	side by side
width	🟢	🔴	🟢
height	🟢	🔴	🟢
margin	🟢	⚪*	🟢
padding	🟢	⚪*	🟢

* vertical margins do not affect surrounding lines

Nesting Elements

- **block elements**
  - can contain **selected** block elements – according to the **specification**
  - can contain any inline (inline-block) elements
- **inline (inline-block) elements**
  - **cannot** contain block elements

```
In <b>bold text <p>paragraph</p> cannot be here</b>.
```

  - can contain any inline (inline-block) elements

```
<b>y = <i>e</i><sup>2<i>x</i></sup></b>
```

y = e^{2x}

Meaning of Elements

- **block elements**
  - usually represent the basic building blocks of a page
  - `header`, `footer`, `section`, `nav`, `h1`, `p`, `div`, …
- **inline (inline-block) elements**
  - usually represent the actual content in the document – text highlighting, images, multimedia, …
  - `b`, `i`, `a`, `img`, `svg`, `video`, `span`, …

- **generic element** (`div`, `span`)
  - has no implicit meaning
  - used for grouping content, usually with a `class` attribute
- **semantic element** (e.g. `nav`, `time`, …)
  - its name clearly expresses its meaning and purpose for the content it wraps
  - recommended to use if available (accessibility, SEO)

HTML Elements Demo

Selected Block Elements

document structure, headings, text blocks, lists, tables, (media, forms)

Basic Content Structure

- **`article`** – article, post (independently usable part of content)
- **`section`** – chapter of an article, post, etc.
- **`aside`** – part that only remotely relates to the main content
- **`header`** – header of document, article, etc.
- **`footer`** – footer
- **`main`** – main content of the page (unique)
- **`nav`** – part containing navigation
- **`div`** – generic block without further meaning (for scripting, styles, etc.)

Basic Content Structure

Headings: h1 - h6

```
<h1>Heading level 1</h1>
<h2>Heading level 2</h2>
<h3>Heading level 3</h3>
<h4>Heading level 4</h4>
<h5>Heading level 5</h5>
<h6>Heading level 6</h6>
```

Heading level 1 HEADING LEVEL 2 HEA

- elements of basic document structure usually require a heading

Text Blocks

```
<p>
  This is a paragraph of text.
  Paragraphs are used
  for regular content.
</p>

<blockquote>
  “This is a quote, displayed
  as a separate block.”
</blockquote>

<address>
  Contact: John Doe
</address>
```

This is a paragraph of text.
Paragraphs are used
for regular content.

“This i
as a se

- closing tag </p> is not required (not recommended)
  - paragraph ends with the start of another block element
- only inline elements are allowed inside p and address

Text Blocks: <details>

```html
<details>
  <summary>Tyrion Lannister</summary>
  <p>Drinks wine like it’s water, talks
</details>

<details open>
  <summary>Hodor</summary>
  <p>Can carry a child, a sled, and the
</details>
```

▶ Tyrion Lannister▼ Hodor

Can carry a child, a s

- **details**
  - can contain any HTML (text, images, lists, forms…)
  - first child must be **summary** – changes state (attribute open)

Preformatted Text

- preserves formatting using spaces and line breaks
- usually in monospaced font
- CSS property: `white-space: pre;`

```
<pre>
  This is preformatted text.
   Preserves    line breaks  and spaces
Tags are still interpreted <b>normally</b>.
</pre>
```

```
  This is preformatted text.
   Preserves    line breaks  and spaces
Tags are still interpreted normally.
```

Ordered and Unordered List

```html
<ul>
  <li>Red team
  <ol>
    <li>Ice Truck Killer</li>
    <li>The Trinity Killer</li>
    <li>The Doomsday Killer</li>
  </ol>
  </li>
  <li>Blue team
  <ol>
    <li>Little Chino</li>
    <li>The Bay Harbor Butcher</li>
  </ol>
  </li>
</ul>
```

- Red team

1. Ice Truck Killer
2. The Trinity Killer
3. The Doomsday Killer

- *unordered list* **`<ul>`** and *ordered list* **`<ol>`** can only contain **`<li>`** elements
- *list item* **`<li>`** can contain any block or inline elements
  - nested lists

Definition List

```
<dl>
  <dt>Ice Truck Killer</dt>
  <dd>
   Brian Moser, known for dismembering
   and arranging bodies.
  </dd>

  <dt>The Doomsday Killer</dt>
  <dd>
  Travis Marshall, murders inspired by
  the biblical Book of Revelation.
  </dd>
</dl>
```

Ice Truck Killer

Brian Moser, known for disme
and arranging bodies.

The Doomsday Killer

Travis Marshall, murders ins
the biblical Book of Revelat

- *definition list* **<dl>** can only contain **<dt>** and **<dd>** elements (in any sequence)
- *definition term* **<dt>** cannot contain sections and headings
- *definition description* **<dd>** can contain any block or inline elements

Table

```html
<table>
  <tr>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Residence</th>
  </tr>
  <tr>
    <td>Jon</td>
    <td>Snow</td>
    <td>Winterfell</td>
  </tr>
  <tr>
    <td>Tyrion</td>
    <td>Lannister</td>
    <td>King's Landing</td>
  </tr>
</table>
```

First Name	Last Name	Residence
Jon	Snow	Winterfell
Tyrion	Lannister	King's Landing

- table **`<table>`** is organized by rows **`<tr>`** (*table row*)
- rows **`<tr>`** contain cells **`<td>`** (*table data*) and **`<th>`** (*table header*)

Table: Cell Merging

```html
<table>
  <tr>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Residence</th>
  </tr>
  <tr>
    <td>John</td>
    <td>Snow</td>
    <td rowspan="2">Winterfell</td>
  </tr>
  <tr>
    <td colspan="2">Hodor</td>
  </tr>
</table>
```

First Name	Last Name	Residence
John	Snow	Winterfell
Hodor		

- **`colspan="n"`** – width in columns
- **`rowspan="n"`** – height in rows
- applies to **`<td>`** or **`<th>`**, not **`<tr>`**

Table: Style

```css
table {
  border: 2px solid blue;
}

td {
  border: 2px solid green;
}

th {
  border: 2px solid red;
}

/* border merging */
table:nth-of-type(2) {
  border-collapse: collapse;
}
```

First Name	Last Name	Residence
Jon	Snow	Winterfell
Tyrion	Lannister	King's Landing

Fi
Jo
Ty

- by default, tables have no visible borders

Table: Semantic Elements

```html
<table>
  <thead>
    <!-- group of rows with column headers -->
  <thead>
  <tbody>
    <!-- group of rows with main content -->
  </tbody>
  <tfoot>
    <!-- group of rows with summary or notes -->
  <tfoot>
</table>
```

- optional elements to emphasize semantics, visually not shown
  - if not specified, browser places all rows in **`<tbody>`** in the DOM
- **`<tfoot>`** can be placed before **`<tbody>`**, browsers will display it below
  - originally (in HTML 4) this optimized table loading

Table: Column Declaration

```html
<table>
  <colgroup style="background-color: yellow;">
    <col
      span="2"
      style="background-color: lightblue">
    <col style="background-color: lightgreen">
    <col>
  </colgroup>
  <tr>
  <!-- table… -->
</table>
```

First Name	Last Name	Residence
Jon	Snow	Winterfel
Tyrion	Lannister	King's La

- used for bulk specification of properties for columns
  - **`<col>`** – one or more columns with shared attributes
  - **`<colgroup>`** – group of columns (structural)
- column definitions affect only structure, not content
- if not defined, the whole table is considered a single column group

Table: Table Caption

```html
<table>
  <caption>
    Game of Thrones Characters
  </caption>
  …
<table>
```

```
Game of Thrones Characters
```

First Name	Last Name	Residence
Jon	Snow	Winterfell
Tyrion	Lannister	King's Landing

```css
/* caption position */
caption {
  caption-side: bottom;
}
```

- **`<caption>`** must be the first child of **`<table>`**
- caption position is above the table by default
  - change using CSS property `caption-side: top|bottom;`

Media: <figure>

```
<figure>
  <img src="tyrion.jpg" alt="Tyrion" height="450">
  <figcaption>Tyrion – he knows things</figcaption>
</figure>
```

- **figure** – block containing media (image, video) with caption
  - images: <img>, <svg>, <canvas>
  - multimedia: <audio>, <video>
  - table: <table>
  - code: <pre><code>
  - another page: <iframe>
- **figcaption** – media caption inside **figure**

Tyrion – he knows things

Forms: <form>

```html
<form>
  <fieldset>
    <legend>Contact Information</legend>

    <label for="name">Name:</label>
    <input type="text" id="name" name="name">

    <label for="email">Email:</label>
    <input type="email" id="email" name="email">

    <input type="submit" value="Send">
  </fieldset>
</form>
```

Contact Information

Name:

Email:

Send

- **form** – contains any flow content, but usually form controls and their labels
  - input, textarea, select + option, button, datalist, output, progress, meter, label
- **fieldset** – groups elements in a form, visually frames them
- **legend** – title for the group, displayed as part of the frame.

Horizontal Separator <hr>

```
Let's make a thick line below this.
<hr>
```

```
hr {
  height: 5px; /* sets thickness. */
  border: none; /* removes default browser borders. */
  background-color: black; /* or background sets line color. */
}
```

Let's make a thick line below this.

Selected Inline Elements

text highlighting, links

Text Highlighting: Semantic Distinction

`<em>`	*emphasized text* – italic with meaning for screen readers
`<strong>`	**important text** – bold, has meaning for screen readers
`<b>`	**bold text** – older, only visual boldness
`<i>`	*italic* – older, only visual italics
`<u>`	underlined text – visual underline
`<mark>`	highlighted text – yellow background
`<small>`	smaller text – smaller font
`<s>`	~~strikethrough text~~ – obsolete or invalid
`<abbr title="">`	WHO – abbreviation with explanation

Text Highlighting: Formulas, Code, Quotes

<var>	x – variable in code or formula
<sub>	H_2O – subscript
<sup>	10^2 – superscript
<code>	`console.log("Code")` – monospace font for code
<dfn>	HTML – new term
<kbd>	Ctrl + C – keyboard input
<q>	*“Short quote”* – inserted quotation marks
<cite>	Author of work – name of source/work
<time>	9 AM – 5 PM
<data value="">	Czech Republic

Links

```
<a href="https://en.wikipedia.org/wiki/Tyrion_Lannister">
  Tyrion Lannister
</a>,
<a href="https://en.wikipedia.org/wiki/Tyrion_Lannister#Character"
   title="Wikipedia" target="_blank">
  Character of Tyrion Lannister
</a>
```

```
                    ,
  Tyrion Lannister   Character of Tyrion Lannister
```

- `target="_blank"` – opens the link in a new window or tab.
- `title` – short description of the link (shown on hover).
- `rel="noopener noreferrer"` – security recommendation for links with target="_blank".

Links: URL

- **to another server**
  - `href="http://www.vutbr.cz/"`
  - `href="http://www.vutbr.cz/doc/info.html"`
- **To the same server**
  - Absolute `href="/doc/info.html"`
  - Relative to current document location
  - `href="next.html"`
  - `href="obsah/clanek.html"`
  - `href="../index.html"`

Links: Anchors

```
<a href="#Character">Character of Tyrion Lannister</a>
…
<p id="Character">
  Drinks wine like it’s water, talks like every word costs gold,
  and somehow survives when everyone else doesn’t.
</p>
```

- `"#Character"` – current document
- `"wiki/Tyrion_Lannister#Character"` – another document
- `"https://en.wikipedia.org/wiki/Tyrion_Lannister#Character"` – another server

Line Breaks

```
Doctor Who<br>
1 TARDIS Lane<br>
Llanfairpwllg<wbr>wyngyllgogerych<wbr>wyrndrobwllllantysiliog<wbr>ogogoch<br>
Anglesey, Wales<br>
UK
```

```
Doctor Who

1 TARDIS Lane

Llanfairpwllgwyngyllgogerychwyrndrobwllllantysiliogogogoch

Anglesey, Wales
UK
```

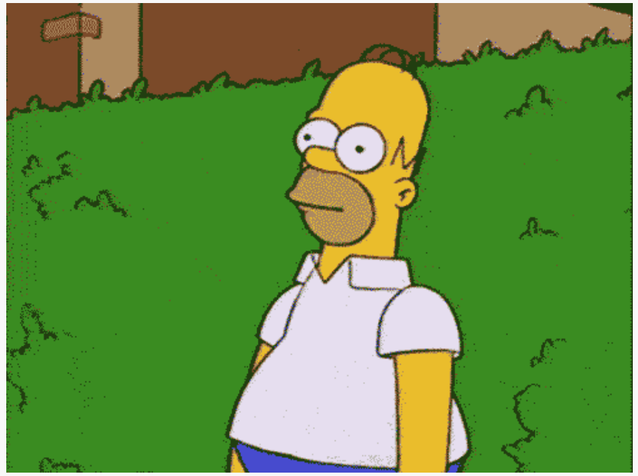
- **
** – hard (forced) line break
- **<wbr>** – optional line break (Word Break Opportunity)
  - does not break unless needed (e.g. for narrow window)

Selected Inline-Block Elements

multimedia, forms

Image <img>

```
<img
  src="homer.gif"
  alt="student when asked to answer a question"
  height="350">
```



- **required attributes**
  - **`src`** – image address (URL or file path)
  - **`alt`** – alternative text shown if the image fails to load, also used by screen readers
- **optional attributes**
  - **`width`**, **`height`** – dimensions in px (better page rendering if browser knows size in advance)
  - `loading` – `eager` = loads immediately, `lazy` = loads when reaches to viewport
  - `srcset`, `sizes` – responsive images
  - `usemap` – reference to image map (<map>)
  - …

Image: Supported Formats

Format	Description	Support
JPEG / JPG	compressed, lossy format for photos	all
PNG	lossless format, supports transparency	all
GIF	animation, 256 colors, transparency	all
WebP	modern format, lossy/lossless, transparency & animation	modern
AVIF	new format, high compression, transparency & HDR	modern
SVG	vector format, scalable without quality loss	modern
APNG	animated PNG	modern

Vector Graphics <svg>

```
<svg width="200" height="200"
   xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 200">
  <!-- Face circle -->
  <circle cx="100" cy="100" r="90" fill="#FFEB3B"
      stroke="black" stroke-width="4" />

  <!-- Eyes -->
  <rect x="58" y="73" width="14" height="14" fill="black" />
  <rect x="128" y="73" width="14" height="14" fill="black" />

  <!-- Smile -->
  <path d="M 60 130 Q 100 170 140 130" stroke="black" stroke-width="6"
    fill="none" stroke-linecap="round" />
</svg>
```

- **width**, **height** – drawing area dimensions
- **viewBox** – coordinate system (minX minY width height)
- **xmlns** – XML namespace (SVG is an XML language – **W3C specification**)

SVG – Example

Raster Graphics <canvas>

```html
<canvas id="myCanvas" width="200" height="200"
    style="border:1px solid #ccc;">
</canvas>

<script>
  const canvas = document.getElementById("myCanvas");
  const ctx = canvas.getContext("2d");
  ctx.lineWidth = 4;

  // Head
  ctx.beginPath();
  ctx.arc(100, 100, 80, 0, Math.PI * 2);
  ctx.fillStyle = "#FFEB3B";
```


- **`width`**, **`height`** – drawing area dimensions
- rendering is done via JavaScript context (**`getContext("<context>")`**):
  - **`2d`** or **`webgl`**
- graphical elements, unlike SVG, **are not represented in the DOM**

SVG – Example

SVG vs. Canvas

Criteria	SVG	Canvas
Type of graphics	🟢 Vector	🟢 Raster
Zoom quality	🟢 Lossless	🔴 Pixelates
Interactivity	🟢 Easy – DOM events (onclick)	🔴 More complex – coordinate calculations
Animation	🟢 CSS, JS libs (GSAP, Anime.js)	⚪ JS code (`requestAnimationFrame`)
Performance	🔴 Weaker with thousands of objects	🟢 Fast for large scenes
Accessibility & SEO	🟢 Visible in DOM	🔴 Only bitmap
Styling	🟢 CSS support (selectors, filters)	⚪ Style only via JS API (`fillStyle`, `strokeStyle`)
Image usage	⚪ Bitmaps possible (`<image>`)	⚪ Bitmaps + pixel manipulation (`drawImage`, `getImageData`)
Typical use	🟢 Icons, charts, maps	🟢 Games, animations, effects
Browser support	🟢 All modern	🟢 All modern

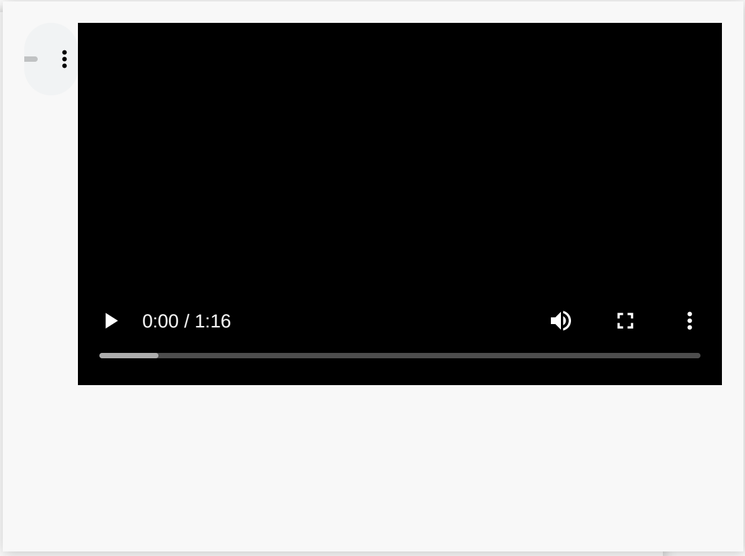
Multiple Image Sources: <picture>

```html
<picture>
  <!-- WebP version for modern browsers -->
  <source srcset="image.webp" type="image/webp">

  <!-- JPG version for smaller screens -->
  <source srcset="image-small.jpg" media="(max-width: 600px)">

  <!-- Default image (fallback) -->
  <img src="image.jpg" alt="Beautiful landscape" width="600">
</picture>
```

- **<source>** – elements specify different versions by:
  - **type** – image format (e.g. image/webp).
  - **media** – conditions via CSS media queries.
- **<img>** – required fallback at the end if no matches.

Audio and Video

```html
<audio style="width: 480px;" controls>
  <source src="frank.mp3" type="audio/mp3">
  <source src="frank.ogg" type="audio/ogg">
  Your browser does not support the audio tag.
</audio>

<video width="480" height="270" controls>
  <source src="shoes.mp4" type="video/mp4">
  <source src="shoes.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

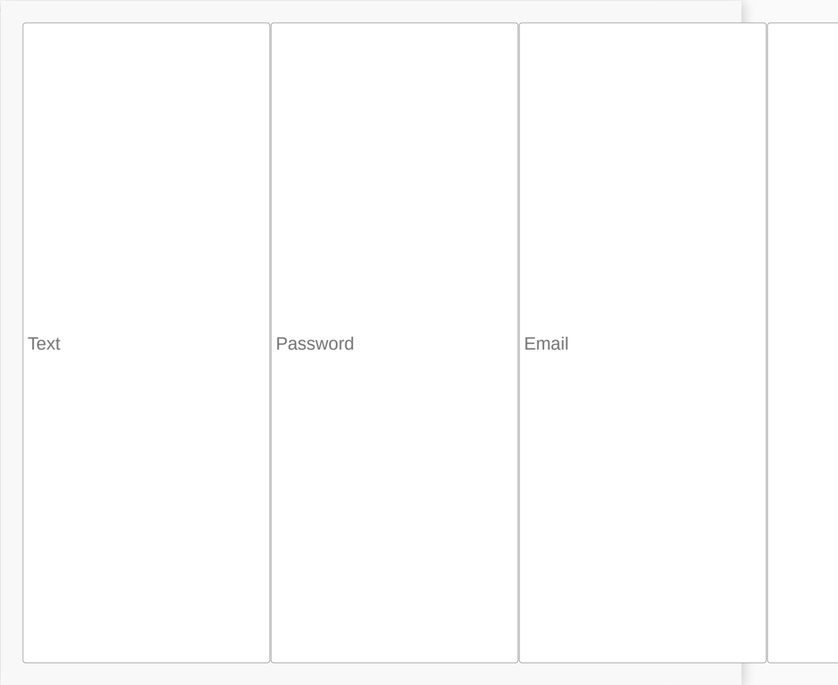


- contains one or more **`<source>`** elements with different formats
  - **audio:** MP3, OGG, WAV, **video:** MP4, WebM, OGV
- **`width / height`** – player dimensions, **`controls`** – shows control panel, **`loop`** – plays in a loop, **`muted`** – starts muted, **`poster`** – image shown before playback, **`autoplay`** – plays after loading (video often requires `muted` due to browsers), **`preload`** – how much data to load before user starts playback

Video – Custom Player

Form Elements: <input>

```html
<input type="text" placeholder="Text"><br>
<input type="password" placeholder="Password"><br>
<input type="email" placeholder="Email"><br>
<input type="number" min="0" max="10"><br>
<input type="url" placeholder="URL"><br>
<input type="tel" placeholder="Telephone"><br>
<input type="date"><br>
<input type="color"><br>
<input type="range" min="0" max="100"><br>
<input type="radio" name="gender" value="male"> Male
<input type="radio" name="gender" value="female"> Female<br>
<input type="checkbox"> Subscribe<br>
<input type="submit" value="Submit">
```



- **`type`** – input type
- `name` – field name, key for data submission, `value` – default field value, `placeholder` – helper text inside field, `readonly` – read-only, `autocomplete` – enable/disable autofill
- **validation**: `required`, `disabled`, `min`, `max`, `step`, `checked`, `maxlength`, `pattern`

Form Elements: <label>, <select>, <textarea>, <button>

```html
<label for="fruit">Choose a fruit:</label>
<select id="fruit" name="fruit">
  <option value="apple">Apple</option>
  <option value="banana">Banana</option>
  <option value="orange">Orange</option>
  <option value="strawberry">Strawberry</option>
</select>
<textarea name="message" rows="4" cols="30"></textarea><br><br
<button type="submit" onclick="send(…)">Send</button>
```

- **`label`** – uses `for` attribute referencing the `id` of the form element it labels
- **`select`** – `multiple` – allows selecting multiple options, `size` – number of visible options without scrolling
- **`textarea`** – `rows` – number of visible text lines, `cols` – number of visible characters per line
- **`button`** – `type`: `submit` – default, submits form; `reset` – clears form; `button` – no action, programmable

Form Example

Status Elements: <meter>, <progress>

```
Battery level: <meter value="0.7">70%</meter>
Disk C usage: <meter id="disk_c" value="2" min="0" max="10">2 out of 10</meter>
Disk D usage: <meter id="disk_d" value="0.6">60%</meter>
<br><br>
Downloading: <progress value="30" max="100">30%</progress>
```

Battery level: Disk C usage: Disk D usage: Downloading:

- **<meter>**
  - displays a known value in a known range
  - useful for measuring levels: temperature, capacity, score, rating
- **<progress>**
  - shows progress of a process
  - useful for downloads, uploads, data loading

Summary

links

Key Information

- concepts: tag, element, attribute, Document Object Model
- HTML Living standard: https://html.spec.whatwg.org/
- validation: https://validator.w3.org/
- normal flow (*normal flow*)
- inline vs. inline-block vs. block elements

- **it is important to try everything practically**
- tutorials, user-friendly manuals:
  - https://www.w3schools.com/html/
  - https://developer.mozilla.org/en-US/docs/Web/HTML

To be continued…

CSS – Introduction to Cascading Style Sheets